



QUENYX

V O P S H U B

INTERNAL

QynVA Guide — Enterprise AI Operator

Quenyx vOPS HUB — v1.0.0 · Document v1.0

Document Version	1.0
Software Version	v1.0.0
Applies To	Quenyx vOPS HUB v1.0.0 — Sprint 25
Classification	Internal
Owner	Platform Engineering
Status	Released
Document Type	Module / platform guide

Table of Contents

	What QynVA is
	Mental model
	Backend
	API
	Capabilities
	Safety & guarantees
	UI

QynVA Guide — Enterprise AI Operator

What QynVA is

QynVA is not a chatbot. It is the **Enterprise AI Operator**: it discovers every module and its capabilities through the AI Adapter Registry, builds a single enterprise context through the Context Engine, reasons through the shared `ModuleAiNarrator`, and proposes **cross-module coordination plans**.

QynVA **never executes anything itself** and **never duplicates module logic**. "Coordination" means an editable, evidence-based plan that references *existing* module actions by their keys; the owning module executes through its own approved, audited path after a human confirms.

Mental model

```
Discover (AI Adapter Registry) → Build context (Context Engine) → Reason (ModuleAiNarrator)
→ Recommend an ordered, editable plan referencing existing module actions
→ Human approves → the owning module executes (QynVA never executes)
```

Backend

Piece	Path
Operator service	<code>app/Services/Platform/Operator/QynVaOperatorService.php</code>
Adapter	<code>app/Services/QuenyxAI/Adapters/QynVaAiAdapter.php</code>
Controllers	<code>app/Http/Controllers/Platform/{Operator,Executive,Analytics,PlatformHealth,EventBus}Contr</code>
Routes	<code>routes/qynva-operator.php</code> (prefix <code>/api/qynva</code>)

API

Base `/api/qynva`. Workspace-scoped, UUID-only. Reads require `accessAi`; the operator requires `can_use_ai`; Platform Health & Event Bus require `administerAi`.

Method	Endpoint	Notes
GET	<code>/operator/capabilities</code>	Discovered modules, capabilities, and the cross-module action catalog.
POST	<code>/operator/operate</code>	Reason over the full enterprise context; return an editable plan + answer.
GET	<code>/executive</code>	Executive Intelligence dashboard (evidence-based).
POST	<code>/executive/summary</code>	Executive AI summary over the deterministic dashboard.
GET	<code>/analytics</code>	Enterprise Analytics (<code>?days=30</code>).
GET	<code>/health</code>	Platform Health snapshot (privileged).
GET	<code>/events</code>	Platform Event Bus introspection (privileged).

Capabilities

`enterprise_operator`, `cross_module_coordination`, `context_engine`, `executive_intelligence`, `platform_analytics`, `platform_health`. The adapter registers in `AppServiceProvider::boot()` like every other module (one line) and is `ai_candidate ⇒ true` in `config/quenyx_ai.php`.

Safety & guarantees

- **Never executes** — plans are advisory and editable; execution is the owning module's job after approval.
- **Evidence-based** — reasons only over the Context Engine output and the real action catalog.
- **Honest** — with AI disabled, the deterministic mock provider answers (flagged `ai_enabled: false`); the evidence stays real.
- **Auditable** — every operator turn records a conversation and publishes a `ConversationCompleted` event on the Platform Event Bus.
- **Workspace isolation, RBAC, UUID-only** throughout.

UI

`/app/workspaces/:id/qynva/operator` — the Operator console (capability discovery + Quenyx AI drawer), with tabs to **Executive Intelligence**, **Enterprise Analytics**, and **Platform Health**.